Kubernetes - Cheat Sheet

Contents

Kubernetes (k8s) is an open-source platform for deploying, scaling, and managing containerized applications. Originally built at Google, modeled on their internal Borg system, open-sourced in 2014. Today it's the de facto standard for running containers in production.

If Docker lets you run one container reliably, Kubernetes lets you run thousands across a cluster of machines while handling failure, rollout, networking, and scaling for you.

1. What Kubernetes Is

The core idea: you describe what you want running ("3 replicas of this app, exposed on port 80, with this config"), and Kubernetes figures out how to make it happen and keep it that way. The model is declarative throughout.

If a node dies, Kubernetes reschedules the pods elsewhere. If an app crashes, it restarts. If traffic spikes, it scales up. You write YAML. The cluster does the work.

2. Architecture

A Kubernetes cluster has two layers.

Control plane (the brain)

Runs on dedicated nodes, or managed by your cloud provider.

Component	Job
API server	The front door, handles all REST requests
etcd	Distributed key-value store, holds all cluster state
Scheduler	Decides which node a new pod runs on
Controller manager	Watches state and reconciles toward desired state
Cloud controller manager	Talks to the cloud provider's API (load balancers, volumes, etc.)

Worker nodes (where your stuff runs)

Component	Job
kubelet	Agent on each node, runs containers as instructed
kube-proxy	Network proxy, handles service routing
Container runtime	Actually runs containers (containerd, CRI-O)

Managed Kubernetes services (GKE, EKS, AKS) hide the control plane from you. You manage only worker nodes, or even those with serverless options like Fargate or GKE Autopilot.

3. Core Objects

Object	What it is
Pod	Smallest deployable unit. One or more containers sharing network and storage.
ReplicaSet	Keeps a stable number of pod replicas running
Deployment	Manages ReplicaSets, handles rollouts and rollbacks
Service	Stable network endpoint for a set of pods
Ingress	HTTP routing into the cluster
ConfigMap	Non-sensitive config data
Secret	Sensitive config data (base64-encoded by default)
Volume	Storage attached to a pod

Object	What it is
PersistentVolume / PersistentVolumeClaim	Cluster-managed persistent storage
Namespace	Logical partition of cluster resources
Job / CronJob	Run-to-completion or scheduled tasks
StatefulSet	Pods with stable identity and persistent storage (DBs, brokers)
DaemonSet	One pod per node (log collectors, monitoring agents)

You declare these in YAML manifests and apply them with `kubectl apply -f file.yaml`.

4. Pods

The smallest unit. Usually one container per pod, but sometimes a few that need to share network and storage (the sidecar pattern).

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: web
  labels:
    app: web
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      ports:
        - containerPort: 80
```

You almost never create raw pods. Use a Deployment instead so failed pods get replaced automatically.

Key facts:

- Pods get a unique IP, but it changes when the pod restarts. Use a Service for stable addressing.
- Containers in the same pod share `localhost` and can share volumes.
- A pod runs on one node. Once scheduled, it stays there until it dies or gets evicted.

5. Deployments

The standard way to run stateless workloads. Manages a ReplicaSet, which manages pods.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: 100m
              memory: 128Mi
            limits:
              cpu: 500m
              memory: 256Mi
```

What it does:

- Keeps 3 pods running. If one dies, a new one spins up.
- Manages rolling updates when you change the image. Old pods drain, new ones come up.
- Handles rollbacks: `kubectl rollout undo deployment/web`.

Selectors and labels link Deployments to pods. The Deployment selects pods with `app: web`. Each pod must carry that label.

6. Services

Pods come and go. Their IPs change. A Service is a stable virtual IP that load-balances across pods matching a selector.

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP
```

Service types:

Type	What it does
`ClusterIP` (default)	Internal-only IP. Reachable from inside the cluster.
`NodePort`	Exposes a port on every node's IP (30000-32767). Good for dev or simple setups.
`LoadBalancer`	Asks the cloud provider for a public load balancer.
`ExternalName`	DNS alias to an external service.

Inside the cluster, other pods reach the service by name. Pods in the same namespace use `web`. Across namespaces, use `web.othernamespace.svc.cluster.local`.

7. Ingress

HTTP and HTTPS routing into the cluster. One entry point, multiple backend services.

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
spec:
  rules:
    - host: api.example.com
      http:
        paths:
          - path: /users
            pathType: Prefix
            backend:
              service:
                name: user-service
                port:
                  number: 80
          - path: /orders
            pathType: Prefix
            backend:
              service:
                name: order-service
                port:
                  number: 80
```

Ingress needs an Ingress controller (nginx-ingress, Traefik, AWS ALB Controller, etc.) running in the cluster. The Ingress resource is just configuration. The controller does the routing.

For modern setups, the Gateway API is the successor. Same idea, more flexible.

8. ConfigMap and Secret

Config data, separate from images. Lets you change config without rebuilding.

ConfigMap (non-sensitive)

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  database_host: postgres.default.svc.cluster.local
  log_level: info
```

Secret (sensitive)

```
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
type: Opaque
stringData:
  username: admin
  password: super-secret-pass
```

`stringData` lets you write plain strings. Kubernetes base64-encodes them on storage. Secrets are base64-encoded for transport. They're not encrypted at rest by default. Enable etcd encryption or use an external secret manager (Sealed Secrets, External Secrets, Vault) for production.

Mounting into a pod

```
spec:
  containers:
    - name: app
      image: myapp:1.0
      envFrom:
        - configMapRef:
            name: app-config
        - secretRef:
            name: db-credentials
```

Or as files:

```
volumes:
  - name: config
    configMap:
      name: app-config
containers:
  - volumeMounts:
      - name: config
        mountPath: /etc/config
```

9. Volumes and Persistence

A Volume lives as long as the pod. For storage that outlives pods, use PersistentVolume (PV) and PersistentVolumeClaim (PVC).

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data-claim
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  storageClassName: standard
```

The PVC asks for storage. A PV satisfies the claim, either provisioned dynamically by the storage class or pre-created by an admin.

Use it from a pod:

```yaml
volumes:
  - name: data
    persistentVolumeClaim:
      claimName: data-claim
containers:
  - volumeMounts:
      - name: data
        mountPath: /var/lib/data
```

Access modes:

- `ReadWriteOnce`: one node can mount it read-write.
- `ReadOnlyMany`: many nodes can mount it read-only.
- `ReadWriteMany`: many nodes can mount it read-write (rare, needs special storage like NFS).

For stateful apps (DBs, brokers), use a StatefulSet so each pod gets its own stable PVC.

10. Namespaces

Logical partitions of a cluster. Useful for separating environments, teams, or apps.

```bash
kubectl create namespace dev
kubectl apply -f app.yaml -n dev
kubectl get pods -n dev
```

Common default namespaces:

- `default`: where things go when you don't specify one.
- `kube-system`: cluster components live here.
- `kube-public`: world-readable cluster info.

- `kube-node-lease`: node heartbeats.

RBAC, network policies, and resource quotas can all be scoped per namespace.

11. Essential kubectl

Command	What it does
`kubectl apply -f file.yaml`	Create or update resources
`kubectl delete -f file.yaml`	Delete resources
`kubectl get pods`	List pods
`kubectl get all`	List most common resources
`kubectl describe pod <name>`	Detailed info, events, recent errors
`kubectl logs <pod>`	View logs
`kubectl logs -f <pod>`	Follow logs
`kubectl logs <pod> -c <container>`	Logs from a specific container
`kubectl exec -it <pod> -- bash`	Shell into a pod
`kubectl port-forward <pod> 8080:80`	Tunnel a local port to a pod
`kubectl rollout status deployment/<name>`	Watch a deployment roll out
`kubectl rollout undo deployment/<name>`	Roll back to the previous version
`kubectl scale deployment/<name> --replicas=5`	Manually scale
`kubectl edit deployment/<name>`	Edit a resource in `$EDITOR`
`kubectl get pods -o wide`	More columns (node, IP)
`kubectl get pods -o yaml`	Full YAML output
`kubectl config use-context <ctx>`	Switch cluster
`kubectl top pods`	CPU and memory usage (needs metrics-server)

Setting the default namespace saves repeated `-n` flags:

```
kubectl config set-context --current --namespace=dev
```

12. Health Checks (Probes)

Three probe types tell Kubernetes about your container's state.

Probe	Purpose	Failure action
`livenessProbe`	Is the container alive?	Kubelet restarts it
`readinessProbe`	Is the container ready to serve traffic?	Service stops sending traffic
`startupProbe`	Has the app finished starting?	Other probes wait until it succeeds

Example:

```yaml
containers:
  - name: app
    image: myapp:1.0
    livenessProbe:
      httpGet:
        path: /health
        port: 8080
      initialDelaySeconds: 30
      periodSeconds: 10
    readinessProbe:
      httpGet:
        path: /ready
        port: 8080
      periodSeconds: 5
    startupProbe:
      httpGet:
        path: /health
        port: 8080
      failureThreshold: 30
      periodSeconds: 10
```

Probe styles:

- `httpGet`: HTTP request to a path.
- `tcpSocket`: TCP connection to a port.
- `exec`: run a command inside the container; non-zero exit means failure.

> **Common mistake**
>
> Pointing the liveness probe at an endpoint that does too much. If the probe hits the DB, a slow DB makes Kubernetes restart your app, making things worse. Keep liveness probes cheap.

13. Scaling

Manual

```
kubectl scale deployment/web --replicas=10
```

Horizontal Pod Autoscaler (HPA)

Scales replicas based on CPU or custom metrics.

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web
  minReplicas: 2
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

HPA scales out when average CPU goes above 70%. Needs metrics-server running in the cluster.

Vertical Pod Autoscaler (VPA)

Adjusts CPU and memory requests and limits per pod. Less common. Conflicts with HPA on the same metric.

Cluster Autoscaler

Adds and removes nodes when the cluster runs out of room or has idle capacity. Cloud provider feature.

14. Rolling Updates

Default deployment strategy. Replaces pods gradually so the service stays available.

```yaml
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1       # extra pods during update
      maxUnavailable: 0 # pods that can be down during update
```

Trigger an update:

```
kubectl set image deployment/web nginx=nginx:1.28
```

Watch it:

```
kubectl rollout status deployment/web
```

Roll back:

```
kubectl rollout undo deployment/web
```

Other strategies:

- `Recreate`: stop all old pods before starting new ones. Causes downtime, but simple.
- Blue-green and canary: usually implemented with extra tooling (Argo Rollouts, Flagger, service mesh).

15. Helm

Helm is the Kubernetes package manager. Bundles related YAML manifests into reusable, parameterized charts.

```
# Install a chart from a public repo
helm install my-postgres bitnami/postgresql

# List installed releases
helm list

# Upgrade
helm upgrade my-postgres bitnami/postgresql --set auth.password=newpass

# Roll back
helm rollback my-postgres 1

# Uninstall
helm uninstall my-postgres
```

A chart contains:

- `Chart.yaml`: metadata.
- `values.yaml`: default config values.
- `templates/`: YAML templates with placeholders that get rendered with values.

Useful for shipping reusable applications (databases, monitoring stacks, CI/CD tools) and for managing your own apps across environments with different value files.

16. Best Practices

- Use Deployments for your apps. They handle pod restarts and rollouts automatically.

- Set resource requests and limits. Without requests, scheduling is guesswork. Without limits, a runaway pod can take down the node.
- Label everything. Selectors and tooling depend on consistent labels.
- Pin image versions. `myapp:1.4.2`, never `myapp:latest` in production.
- Use namespaces to isolate environments. Don't mix dev and prod in the same namespace.
- Probe your apps. Liveness, readiness, startup. They keep traffic away from unhealthy pods.
- Store config in ConfigMaps, secrets in Secrets. Don't bake them into images.
- Manage YAML with Git. Treat manifests like code. Kustomize and Helm help.
- Use RBAC. Least-privilege service accounts for apps and humans.
- Plan for failure. Nodes die, pods get evicted. Your app should handle restarts gracefully.

17. Common Gotchas

- No requests means default scheduling. Pods without requests can land anywhere and crowd a node. Set them.
- Limits cause OOM kills. If a container hits its memory limit, the kernel kills it. Set realistic limits or you'll get random restarts.
- `ClusterIP` services aren't reachable from outside. Use `NodePort`, `LoadBalancer`, or Ingress.
- DNS uses the service name only within the namespace. Cross-namespace needs `name.namespace.svc.cluster.local`.
- Secrets aren't encrypted by default. They're base64-encoded. Enable etcd encryption or use an external secret manager.
- `kubectl delete` cascades by default. Deleting a Deployment also deletes its pods. Sometimes that's not what you want.
- PVCs may not delete with their pods. Depending on the reclaim policy, the underlying storage can survive (good) or disappear (bad).
- Probes can cascade failures. A flaky DB making liveness probes fail can restart all your app pods, making the outage worse.
- `kubectl apply` is declarative but tracks changes via annotations. Editing resources outside Git can confuse the diff.
- The Kubernetes API has versioning. `apiVersion: apps/v1` is current for Deployments. Older versions like `extensions/v1beta1` are long gone.
- Resource limits apply per container. A 2-container pod with a 1Gi limit each can use 2Gi total.

18. Quick Reference

Concept	Key fact
Pod	Smallest unit, one or more containers sharing network and storage
Deployment	Manages stateless pods, handles rollouts

Concept	Key fact
Service	Stable virtual IP for a set of pods
Ingress	HTTP routing into the cluster
ConfigMap	Non-sensitive config
Secret	Sensitive config (base64-encoded, not encrypted by default)
PVC	Persistent storage request
Namespace	Logical partition
HPA	Autoscale pods based on CPU or custom metrics
Liveness probe	Restarts the container if it fails
Readiness probe	Stops sending traffic if it fails
Helm	Package manager for Kubernetes
`kubectl apply`	Create or update from YAML
`kubectl logs`	View container output
`kubectl describe`	Detailed info, events, errors
`kubectl rollout undo`	Roll back to the previous version